

L41: SimPy–Gymnasium Wrapper

Simulation Without a Black Box

The OpenAI Gymnasium API

Core Contract

Any environment must implement:

- `reset()` $\rightarrow$ (obs, info): start a new episode
- `step(action)` $\rightarrow$ (obs, reward, terminated, truncated, info)
- `observation_space`: shape and dtype of observations
- `action_space`: set of valid actions

```
import gymnasium as gym
```

```
env = gym.make("CartPole-v1")  
obs, info = env.reset(seed=0)  
for _ in range(200):  
    action =  
         $\hookrightarrow$  env.action_space.sample()  
    obs, rew, term, trunc, info \  
        = env.step(action)  
    if term or trunc:  
        obs, info = env.reset()  
env.close()
```

Why this matters: every RL library (stable-baselines3, RLlib, CleanRL) expects this exact interface. Build to it once; use any algorithm.

The Synchronization Challenge

Problem: SimPy is driven by an internal event loop, while Gymnasium expects the agent to call `step()` from outside.

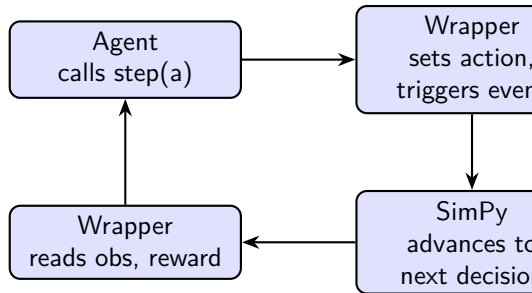
Conflict:

- SimPy wants to advance time to the *next event* continuously
- The agent wants to *pause* SimPy at each decision epoch, observe state, choose an action, and resume

Solution: coroutine-based handoff

Use

`simpy.Environment.run(until=event)` to stop at a specific event; signal that event from the agent via a SimPy Event.



simdes.envs.QueueGymEnv Architecture

```
class QueueGymEnv(gymnasium.Env):
    def __init__(self, arrival_rate, service_rate, max_servers=3, seed=None):
        super().__init__()
        self.lam = arrival_rate; self.mu = service_rate
        self.observation_space = spaces.Box(
            low=np.array([0, 0.0]),          # [queue_len, utilisation]
            high=np.array([50, 1.0]),
            dtype=np.float32)
        self.action_space = spaces.Discrete(max_servers)  # c in {0,1,2}
        self._rng = np.random.default_rng(seed)

    def reset(self, *, seed=None, options=None):
        self.env = simpy.Environment()
        self.server = simpy.Resource(self.env, capacity=1)
        self._total_wait = 0.0; self._n_served = 0
        self._action_event = self.env.event()
        self.env.process(self._arrival_process())
        self.env.run(until=self._action_event)  # run to first decision
        return self._obs(), {}
```

Observation Space and Action Space Design

Observation Space

What the agent can observe at each decision epoch:

- **Queue length** $q \in \{0, \dots, 50\}$
- **Server utilisation** $\hat{\rho} \in [0, 1]$
- **Time of day** (optional, for non-stationary demand)
- **Inventory level** (for combined queue-inventory problems)

Keep the observation vector *small* — only include variables the decision maker could realistically observe.

Action Space

spaces.Discrete(n) — one integer per action:

- **Routing:** $\{0, 1\}$ = server 1 or 2
- **Staffing:** $\{1, 2, 3\}$ = number of servers open
- **Inventory:** $\{0, 5, 10, \dots, 50\}$ = order quantity

Warning

Avoid action spaces larger than necessary — the RL algorithm must explore all actions. Prune infeasible actions from the space.

Reward Shaping: Designing the Learning Signal

Common reward definitions:

- $r_t = -W_{q,t}$: negative wait time of the customer served in step t
- $r_t = -(\text{wait cost} + \text{staffing cost})$: combined cost
- $r_t = +1$ if customer served, 0 if balked: throughput incentive

Reward shaping:

Adding auxiliary rewards to speed up learning, e.g., small reward for keeping utilisation in $[0.6, 0.85]$.

Reward Hacking Risk

Shaped rewards must align with the true objective. An agent rewarded for high throughput may admit very long queues. Always evaluate the *true* objective (e.g., $\mathbb{E}[W_q]$) separately from the shaped reward.

Discount Factor

Use $\gamma \approx 0.99$ for long horizons. Smaller γ makes the agent myopic (ignores future customers).

Training Loop with stable-baselines3 PPO

```
from stable_baselines3 import PPO
from simdes.envs import QueueGymEnv

env = QueueGymEnv(arrival_rate=0.8, service_rate=1.0, seed=0)

model = PPO(
    policy="MlpPolicy",
    env=env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    gamma=0.99,
    verbose=1,
    seed=42,
)
model.learn(total_timesteps=200_000)
model.save("ppo_queue")

# Evaluate
from stable_baselines3.common.evaluation import evaluate_policy
mean_r, std_r = evaluate_policy(model, env, n_eval_episodes=100)
print(f"Mean return: {mean_r:.2f} +/- {std_r:.2f}")
```


Evaluation: Mean Episode Reward over 100 Episodes

Evaluation protocol:

1. Fix evaluation seeds (separate from training seeds)
2. Run 100 episodes with the learned policy (deterministic: $\varepsilon = 0$)
3. Compute mean and std of episode return
4. Compare to at least one baseline (e.g., threshold policy)

What to report:

- Mean episode reward ± 1 std
- Mean W_q (operational KPI, not just RL reward)
- Learning curve: episode return vs. timestep

Common Mistake

Evaluating the policy with the same seed used during training over-estimates performance — the agent may have memorized that trajectory. Always use held-out evaluation seeds.

Sanity Check

If the learned policy is worse than a simple threshold rule, something is wrong with reward design, observation space, or hyperparameters.

Key Takeaways

L41 Summary

1. The Gymnasium API (`reset()`, `step()`, `spaces`) is the universal interface for RL environments — build to it once.
2. Wrapping SimPy requires a coroutine handoff: use `env.run(until=event)` to pause SimPy at decision epochs.
3. Keep observation and action spaces minimal — only expose what the real decision maker can observe and control.
4. Reward shaping accelerates learning but must not misalign with the true objective; always evaluate the true KPI separately.
5. Use held-out evaluation seeds and compare to at least one heuristic baseline before claiming success.

Next: L42 — Deep RL: DQN and PPO for large state spaces.